

UNITAT 2 · SETMANA 2

Crides al sistema i E/S de baix nivell

📅 Setmana 2 · dates per confirmar </> Laboratori TD1 (resolt + repte IA) ~ 4 animacions

MANUAL M2 · TEORIA DE LA UNITAT

Crides al sistema i E/S de baix nivell

Lectura bàsica de la unitat 2 (≈ 1,5 h). Cada secció acaba amb un enllaç **Practica-ho** → a l'exercici o l'animació que la il·lustra. Les xifres surten d'execucions reals dels programes del TD1 a la màquina del curs (Linux 7.0, x86-64, glibc 2.43).

CONTINGUT

- | | | | |
|----|---|----|---|
| §1 | Mode usuari i mode nucli | §4 | E/S de baix nivell amb <code>read</code> i <code>write</code> |
| §2 | Anatomia d'una crida al sistema | §5 | Quant costa una crida |
| §3 | Observar: <code>man</code> , <code>strace</code> , <code>objdump</code> | §6 | Més enllà: <code>vDSO</code> , <code>io_uring</code> , altres arquitectures |

§1 Mode usuari i mode nucli

El processador té almenys dos **nivells de privilegi**. A x86-64 s'anomenen anells: el nucli s'executa a l'anell 0 (CPL 0) i els programes a l'anell 3 (CPL 3). En mode usuari, les instruccions que toquen el maquinari (entrada/sortida, la taula de pàgines, desactivar interrupcions) estan prohibides: si un programa les intenta, el processador llança una excepció i el nucli l'acaba.

Aquesta frontera és la base de tota la resta: gràcies a ella un programa no pot llegir la memòria d'un altre, escriure al disc saltant-se els permisos ni apoderar-se de la CPU. Quan un programa necessita una cosa que només pot fer el nucli, **la demana**: això és una crida al sistema.

§2 Anatomia d'una crida al sistema

Quan el teu programa escriu `write(1, "hola\n", 5)`, en realitat crida una funció de glibc (l'*embolcall*) que segueix la convenció de Linux x86-64:

1. Posa el **número de la crida** al registre `rax` (1 per a `write`, 0 per a `read`, 110 per a `getppid`) i els arguments a `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. Les dades no viatgen als registres: `rsi` porta l'adreça del text.
2. Executa la instrucció `syscall`: la CPU canvia a mode nucli i salta a una adreça fixa que el nucli va configurar en arrencar (`entry_SYSCALL_64`). El programa no pot triar on salta.
3. El nucli fa servir `rax` com a índex a `sys_call_table` i executa el gestor (`ksys_write`), que valida el descriptor, copia les dades de la memòria del programa (`copy_from_user`) i crida el controlador del dispositiu.
4. El resultat torna a `rax` i la instrucció `sysret` retorna la CPU a mode usuari.

5. Si `rax` és entre `-4095` i `-1`, l'embolcall ho tracta com un error: desa el codi a `errno` i retorna `-1`. Per això el patró de tot el curs és `exit_if(rc == -1, "...")`, que imprimeix l'error amb `perror`.

Sortida real

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -A4 -E '^[0-9a-f]+ <getppid@>'
000000000000ff710 <getppid@Glibc_2.2.5>:
    ff710:    endbr64
    ff714:    mov     $0x6e,%eax
    ff719:    syscall
    ff71b:    ret
```

Practica-ho → animació A2.1 · El viatge d'una crida al sistema

§3 Observar: `man`, `strace`, `objdump`

Tres eines per investigar qualsevol programa sense el seu codi font:

- `man 2 write` documenta la crida al sistema; `man 3 printf`, la funció de biblioteca; `man 3p`, l'estàndard POSIX.
- `strace ./programa` mostra cada crida al sistema amb els arguments i el resultat; `-f` segueix els fills, `-r` hi afegeix el temps entre crides i `-c` en fa un resum.
- `objdump -d` desassembla: així es veu la instrucció `syscall` dins de `libc`.

Una sorpresa en fer servir `strace`: l'`echo` del TDI fa una **trentena de crides abans d'arribar a `main`**. Són del carregador dinàmic, que obre `libc.so.6`, la mapa a la memòria amb `mmap` i en protegeix les pàgines amb `mprotect`.

Practica-ho → animació A2.2 · `strace` en directe

§4 E/S de baix nivell amb `read` i `write`

Cada procés neix amb tres **descriptors de fitxer** oberts: 0 (entrada estàndard), 1 (sortida estàndard) i 2 (error estàndard). El shell decideix cap a on apunten: `< fitxer`, `> fitxer`, `2>&1` o una canonada. El programa no ho sap: només fa servir els números.

CRIDA	RETORNA	COMPTE AMB
<code>read(fd, buf, n)</code>	bytes llegits (1...n), 0 al final del fitxer, -1 en cas d'error	pot retornar-ne <i>menys</i> que <code>n</code> (canonada, terminal, xarxa, senyal)
<code>write(fd, buf, n)</code>	bytes escrits (1...n), -1 en cas d'error	escriptura parcial (canonada o sòcol ple); <code>EINTR</code> si un senyal la interromp

Per això els programes del curs repeteixen: `read_exact` llegeix fins a completar la petició i `write_all` escriu fins que surt tot. A més, `read` i `write` copien **bytes**, no text: escriure un `long` en copia els 8 bytes en l'ordre de la màquina (*little-endian* a x86-64), i escriure una estructura també en copia el farciment d'alineació (24 bytes per a `struct nopad`).

Sortida real

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00
0000016
```

⚠ Error clàssic: avançar el punter el que falta

L'exemple de lectura de les diapositives originals (B25, diapositiva 51) feia `length = length - byte; buf = buf + length;`. Amb un fitxer normal funciona —la primera lectura ho porta tot i `length` val 0—, però amb una canonada o un sòcol deixa forats i escriu **fora de la memòria intermèdia**. La manera correcta és `buf += byte`, tornant a provar amb `EINTR` i parant amb 0. Mira-ho pas a pas a A2.4 i prova-ho amb `cat file | ./read_struct`.

Practica-ho → TD1, exercicis 3 a 7

§5 Quant costa una crida

Cada anada i tornada al nucli té un cost fix: canviar de mode, desar i restaurar registres, validar arguments. El programa `bench_read` del TD1 el mesura copiant 8 MiB a `/dev/null` amb memòries intermèdies d'1 byte a 1 MiB. Aquests són els resultats reals a la màquina del curs:

Sortida real

```
$ ./bench_read      # 8 MiB file, buffers of 1 B to 1 MiB
buffer_bytes,read_calls,seconds,MB_per_s,ns_per_iteration
1,8388609,6.4984,1.3,775
4,2097153,1.5997,5.2,763
16,524289,0.3952,21.2,754
64,131073,0.0998,84.1,761
256,32769,0.0260,322.6,794
1024,8193,0.0075,1121.4,913
4096,2049,0.0025,3351.2,1222
16384,513,0.0010,8319.5,1966
65536,129,0.0008,10834.7,6002
262144,33,0.0007,11832.2,21484
1048576,9,0.0008,10287.9,90598
```

Amb 1 byte per crida hi ha 8,4 milions d'iteracions i la còpia triga segons; amb 64 KiB, 129 iteracions i menys d'un mil·lisegon. Ajustar el model $temps = crides \times c_0 + bytes \times c_1$ a les mesures dona un peatge d'uns **0,7 µs per iteració** (`read` + `write`) i menys de 0,1 ns per byte copiat. Conclusió pràctica: fes servir memòries intermèdies de pocs KiB o més (la biblioteca de C ho fa per tu amb `printf` / `fwrite`, unitat 4) i no llegeixis mai byte a byte en un bucle.

Practica-ho → animació A2.3 · El cost d'una crida al sistema

§6 Més enllà: vDSO, `io_uring` i altres architectures

- **vDSO.** Algunes «crides» molt freqüents, com `clock_gettime`, no entren al nucli: Linux mapa a cada procés una petita biblioteca (vDSO) que llegeix l'hora d'una pàgina compartida. Per això no apareixen a `strace`.
- **`io_uring`** (Linux 5.1, 2019). Per a programes que fan molta E/S: el programa i el nucli comparteixen dues cues a la memòria; es poden enviar centenars d'operacions amb una sola crida, o cap. La mateixa idea que la memòria intermèdia gran, aplicada a les crides.
- **Altres architectures.** A ARM de 64 bits la instrucció és `svc #0` i el número va a `x8`; a RISC-V és `ecall` amb el número a `a7`. Els números canvien entre architectures; la idea, no.
- **Observabilitat amb eBPF.** Eines com `bpftrace` compten crides a tot el sistema amb un cost molt menor que `strace`, que atura el procés a cada crida.